

Efektywna reprezentacja słownika

Zaawansowane Biblioteki Programistyczne

Piotr Marcol

31.03.2026

- Reprezentacja dużego zbioru słów w sposób efektywny
- Szybkie sprawdzanie przynależności słowa do słownika
- Minimalizacja zużycia pamięci
- Możliwość dalszych rozszerzeń (np. podpowiedzi, autouzupełnianie)

- Duże słowniki (setki tysięcy słów)
- Powtarzające się prefiksy
- Naiwne przechowywanie (np. lista) jest nieefektywne
- Potrzeba szybkiego wyszukiwania

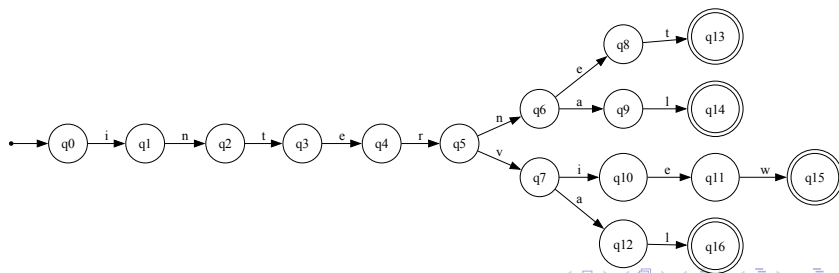
- Tablica / lista słów
 - prosta implementacja
 - wolne wyszukiwanie
- Drzewo prefiksowe (Trie)
 - współdzielenie prefiksów
 - szybkie operacje
- Minimalny automat deterministyczny (DAFSA)
 - minimalna liczba stanów dla danego zbioru słów
 - bardzo wysoka efektywność

Drzewo prefiksowe (Trie)

- Każda krawędź odpowiada literze
- Ścieżka od korzenia tworzy słowo
- Wspólne prefiksy są współdzielone

Przykład:

- interview, internet, internal, interval
- Wspólny prefiks: inter
- Rozgałęzienie po różnych sufiksach

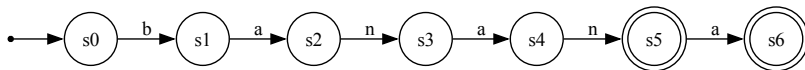


Stany akceptujące

- Nie każdy węzeł oznacza słowo
- Węzły końcowe są oznaczane jako **akceptujące**

Przykład:

- banan — stan akceptujący
- banana — inny stan akceptujący
- Jeden prefiks może być pełnym słowem i częścią innego



- Trie można traktować jako automat deterministyczny
- Każdy węzeł = stan
- Każda krawędź = przejście

Problem:

- duża liczba stanów

Rozwiązanie:

- minimalizacja automatu

- Łączenie stanów o identycznych przejściach
- Identyczne sufiksy $\rightarrow$ jeden podgraf
- Proces od liści do korzenia
- Efekt:
 - brak duplikacji końcówek
 - minimalna liczba stanów

Minimalny automat (DAFSA)

Definicja

DAFSA to minimalny deterministyczny automat akceptujący skończony zbiór słów

- Deterministyczny acykliczny automat skończony
- Łączy identyczne poddrzewa
- Eliminuje redundancję

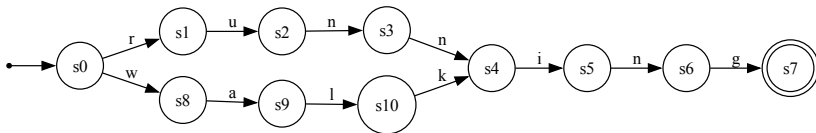
Efekt:

- mniej stanów
- mniejsze zużycie pamięci

- koszt budowy automatu
- trudniejsza implementacja niż Trie
- brak łatwej modyfikacji po zbudowaniu

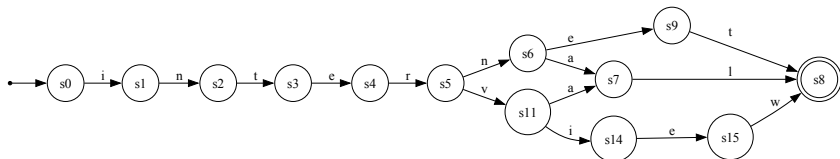
Przykład optymalizacji

- Słowa:
 - running
 - walking
- Wspólny sufiks: ing
- W DAFSA:
 - końcówka ing przechowywana tylko raz



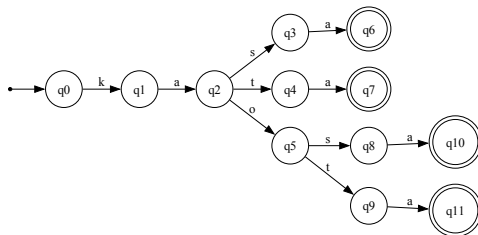
Przykład optymalizacji (cd.)

- Słowa:
 - interview, internet, internal, interval
- Wspólny prefiks: inter
- Wspólny sufiks: al dla *internal* i *interval*
- DAFSA łączy zarówno wspólne prefiksy, jak i sufiksy

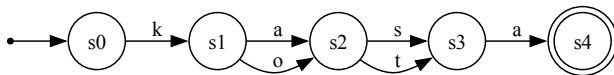


Przykład optymalizacji (cd. 2)

- Słowa: kasa, kata, kosa, kota
- Trie (12 stanów, 11 krawędzi):



- Po minimalizacji (5 stanów, 6 krawędzi):



- **Trie**

- optymalizuje wspólne prefiksy
- prostsza implementacja

- **DAFSA**

- optymalizuje prefiksy i sufiksy
- minimalna liczba stanów

- **Najlepsze wyniki:**

- dane z powtarzalnymi fragmentami (prefiksy i/lub sufiksy)

- Wyszukiwanie: $O(m)$
- Budowa:
 - Trie: $O(\text{sumy długości słów})$
 - DAFSA: dodatkowy koszt minimalizacji
- Pamięć:
 - Trie: większa
 - DAFSA: znacznie mniejsza

Dlaczego nie lista słów?

- Lista:
 - wyszukiwanie: $O(n \times m)$
 - duża redundancja danych
- Trie / DAFSA:
 - wyszukiwanie: $O(m)$
 - współdzielenie struktur
 - oszczędność pamięci

- Autouzupełnianie
- Wyszukiwanie prefiksowe
- Spell-check
- Analiza językowa
- Kompresja słowników

- Budowa Trie ze zbioru słów
- Konwersja do minimalnego automatu
- Interfejs do zapytań:
 - sprawdzenie przynależności słowa
 - zapytania prefiksowe
- Reprezentacja jako graf stanów

- Biblioteka w języku C++
- Reprezentacja słownika jako minimalizowany automat deterministyczny
- Wejście:
 - posortowany zbiór słów (porządek leksykograficzny)
- Budowa:
 - inkrementalne dodawanie słów
 - częściowa minimalizacja w trakcie budowy
 - finalizacja minimalizacji (`finish()`)
- Operacje:
 - sprawdzanie przynależności słowa
 - zapytania prefiksowe

- Trie — prostota i czytelność
- DAFSA — optymalność pamięciowa
- Automaty — formalne i wydajne podejście
- Możliwość zastosowań w NLP i systemach wyszukiwania

- słowniki i spell-checkery
- autouzupełnianie (autocomplete)
- systemy wyszukiwania
- kompresja danych tekstowych

- Implementacja struktury
- Testy na dużych zbiorach danych
- Analiza wydajności
- Wizualizacja automatu

Dziękuję za uwagę!

Efektywna reprezentacja słownika

Zaawansowane Biblioteki Programistyczne

Piotr Marcol

31.03.2026